



Chapter 3: The Windows App SDK for a WPF Developer

In the previous chapter, we introduced a wide range of topics – from XAML basics to binding to user controls. It was one of the biggest chapters of the book, since WinUI is significantly different from Windows Forms when it comes to building the **user interface (UI)**.

WinUI, on the other hand, has a familiar feeling for **Windows Presentation Foundation (WPF)** developers, since it shares many similarities. The most important one, without any doubt, is the UI layer – both technologies are based on XAML. This means that you'll be able to reuse all your existing knowledge when you modernize the UI of your applications.

However, what's the point of a new technology if it doesn't introduce anything new? In this chapter, we're going to learn the main differences between WPF and WinUI so that you can plan accordingly the migration phase of your application.

We'll cover the following topics:

- Importing XAML namespaces
- The new **x:Bind** markup expression for binding
- Using the dispatcher
- Localizing your application to support multiple languages

By the end of this chapter, you'll have a deeper knowledge of the differences between WPF and WinUI so that you can start planning the

migration of your existing applications.

Technical requirements

The code for this chapter can be found at the following link:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter03>

Importing XAML namespaces

One of the basic features of XAML is the ability to import namespaces so that you can use controls or classes that aren't included in a basic framework but come from a class that belongs to a custom namespace (it could be part of your project, an external class library, or a NuGet package). Namespaces are declared at the top-level container, typically the **Window** class or the **Page** class.

In WPF, you use the following syntax to declare a namespace:

```
<Window x:Class="MyApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/
            xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/
            xaml"
        xmlns:d="http://schemas.microsoft.com/expression/
            blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/
            markup-compatibility/2006"
        xmlns:internal="clr-namespace:
            MyApp.MyInternalNamespace"
        xmlns:external="clr-namespace:MyExternalLib
            .MyNamespace;assembly=MyExternalLib">
```

```
Title="MainWindow" Height="450" Width="800">
</Window>
```

The previous code snippet shows the declaration of two types of namespaces:

- **Internal:** This means that the namespace belongs to the same project that contains the application.
- **External:** This means that the namespace belongs to an external assembly, such as a class library or a NuGet package.

WinUI has simplified the namespace declaration by removing the differences between internal and external namespace. This is how the same namespace definition looks in WinUI:

```
<Window x:Class="MyApp.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/
    xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/
    xaml"
  xmlns:d="http://schemas.microsoft.com/expression/
    blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/marku
p-
  compatibility/2006"
  xmlns:internal="using:MyApp.MyInternalNamespace
"
  xmlns:external="using:MyExternalLib.MyNamespace
">
  Title="MainWindow" Height="450" Width="800">
</Window>
```

The other difference is that the **clr-namespace** prefix has been replaced by the **using** one to match the same C# syntax.

There are no differences though in the way you use the namespace inside your XAML code. Let's say, for example, that your external library

exposes a control called **MyCustomControl**. To use it in your XAML code, you just have to add the namespace name as a prefix:

```
<external:MyCustomControl />
```

Binding with x:Bind

Binding is, without any doubt, the most powerful feature of XAML. Thanks to binding, you can easily connect the UI with data, keeping the two layers separated. As we're going to see in [*Chapter 6, Building a Future-Proof Architecture*](#), binding is at the core of the **Model-View-ViewModel (MVVM)** pattern, which is the most productive technique to build applications that are easy to test, maintain, and evolve.

Traditional binding in WPF, however, has a few limitations, caused by the fact that it's always evaluated at runtime. This approach introduces challenges such as the following:

- Performance issues.
- You cannot catch binding errors until an application is running.
- If you want to change the way data is displayed in a UI, you must rely on a converter, which introduces additional overhead.

With the goal of tackling these problems, Microsoft has introduced in WinUI a new markup expression called **x:Bind**. It works in a similar way to traditional binding, but it brings a huge change under the covers – the binding expression isn't evaluated anymore at runtime but at compile time. This approach has the following benefits:

- It improves performance, since bindings are now compiled together with the rest of the code of your application.
- It simplifies error management. Since the binding expression is compiled into code, errors are immediately caught during the build process.

- It opens the opportunity to create a binding channel not only with regular data but also with a function.

Let's see a basic binding sample:

```
<TextBlock Text="{Binding Path=MyText}" />
```

If we convert this expression using **x:Bind**, we need to use the following syntax:

```
<TextBlock Text="{x:Bind=MyText}" />
```

Looks easy! However, compared to WPF, there are a few key differences. Let's explore them!

Different context

The traditional binding markup expression is deeply connected with the **DataContext** property exposed by all the XAML controls. Thanks to this property, you can define which is the binding context, thus the control (and all its children) can access all the properties that are exposed by that context. If this concept isn't fully clear to you, you can read [*Chapter 2, The Windows App SDK for a Windows Forms Developer*](#) (if you skipped it), where we introduced the basic XAML concepts, including binding.

In WinUI, however, since the **x:Bind** expression is compiled together with the rest of the code, you can't use **DataContext** as the default source, but it automatically uses the page or the user control itself. This means that a binding expression created with the **x:Bind** expression will look for properties, fields, and methods in the code-behind class. To better understand this behavior, try to declare the following property in the code-behind of one of your pages in a WinUI application:

```
public string Name { get; set; }
```

Now, move to the XAML file and add a **TextBlock** control that uses the **x:Bind** expression:

```
<TextBlock Text="{x:Bind Name}" />
```

You will notice that, as soon as you type the **x:Bind** expression, the IntelliSense provided by Visual Studio will trigger and show you all the properties exposed by the code-behind class, including your new **Name** property.

However, in a real application, this isn't a good example. Even if you're going to change the value of the **Name** property during the execution, you won't see the update in the UI because **Name** is a simple property that doesn't implement the **INotifyPropertyChanged** interface. As such, it isn't able to notify the target of the binding channel (the **Text** property of the **TextBlock** control) that the value has changed.

A better example is when you adopt the MVVM pattern, which relies on binding to connect the UI controls in your page to the properties and commands exposed by your ViewModel class. How do we achieve the same with the new **x:Bind** expression? Using the code-behind class as the ViewModel isn't an acceptable solution, since it would defeat the benefit of using the MVVM pattern to split the UI layer from the business logic. The code-behind class is a special class that is coupled with the XAML page; you can't, for example, store it in a different class library, as you can do with a ViewModel.

The solution is to declare a ViewModel instance as a property in your class. For example, let's say that your view, called **MainPage**, is connected to a ViewModel, called **MainPageViewModel**. In this case, you would need to declare a property of the **MainPageViewModel** type in your code-behind:

```
public MainPageViewModel ViewModel { get; set; }
```

Then, in the public constructor of the code-behind class, you must add the code to initialize the ViewModel. There are multiple options to achieve this goal, based on the way you are implementing the MVVM, such as the following:

- Manually create a new instance of the ViewModel.
- If you're using **Dependency Injection (DI)**, retrieve an instance of the ViewModel from the container.

You can also keep setting the **DataContext** property in the same way as you were doing in WPF. For example, let's say that you are setting the ViewModel directly in XAML, as shown in the following example:

```
<Page x:Class="MyApp.MainPage"
      DataContext="{Binding Source={StaticResource
          ViewModelLocator}, Path=MainPageViewModel}">
  <!-- Page content -->
</Page>
```

You can set the **ViewModel** property in code-behind using the following approach:

```
public MainPageViewModel ViewModel { get; set; }
public MainWindow()
{
    this.InitializeComponent();
    ViewModel = this.DataContext as MainPageViewModel;
}
```

Once you have initialized the ViewModel property, you can use it with the **x:Bind** markup expression, thanks to support for property paths, which means that you can simply use a dot (.) to navigate through the properties exposed by your ViewModel. For example, let's say that ViewModel exposes a property called **Name**, as shown in the following example:

```
public class MainPageViewModel: ObservableObject
```

```
{
    private string _name;
    public string Name
    {
        get { return _name; }
        set { SetProperty(ref _name, value); }
    }
}
```

If you want to connect it to a UI control, you can use the following syntax:

```
<TextBlock Text="{x:Bind Path=ViewModel.Name}" />
```

Default binding mode

In WPF, by default, binding is created in **OneWay** mode. As such, in the earlier example, every time the value of the **Name** property changes, the control will update itself to display the new value (as long as the **Name** property implements the **INotifyPropertyChanged** interface).

When you use the **x:Bind** expression, the default binding mode is **OneTime** instead, which means that the binding will be evaluated only once when the channel is created. In many cases, this new default approach helps to save memory and improve performance, since there are many scenarios where you don't need to keep the UI constantly in sync. However, the consequence is that, if the **Name** property changes, the UI won't be updated to reflect it. As such, if this is your scenario, it's important to specify the **OneWay** mode:

```
<TextBlock Text="{x:Bind=MyText, Mode=OneWay}" />
```

Using x:Bind to define a DataTemplate

A quite common scenario in which binding is used is defining **DataTemplate**. As mentioned in the previous chapter, **DataTemplate** is used to shape the look and feel of data that is displayed by a control. For example, if you're displaying a data collection using a control such

as **ListView** or **GridView**, **DataTemplate** defines the appearance of a single item in the list.

In this context, binding is used to connect the various elements of **DataTemplate** with the properties of an item that you want to display. For example, let's say you want to display a collection of **Person** objects, which is defined as follows:

```
public class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public DateTime BirthDate { get; set; }
}
```

The **DataTemplate** object that can be used to display this collection might look like this:

```
<DataTemplate x:Key="PersonTemplate">
    <StackPanel>
        <TextBlock Text="{Binding Path=Name}" />
        <TextBlock Text="{Binding Path=Surname}" />
        <TextBlock Text="{Binding Path=BirthDate}" />
    </StackPanel>
</DataTemplate>
```

The only required step when switching to **x:Bind** is to specify the type of data that **DataTemplate** refers to. Remember that when you use **x:Bind**, the binding channels are compiled, so Visual Studio needs to know which type of data you're trying to visualize. This goal is achieved by setting the **x:DataType** template of the **DataTemplate** object, as shown in the following example:

```
<DataTemplate x:Key="PersonTemplate"
    x:DataType="local:Person">
    <StackPanel>
        <TextBlock Text="{x:Bind Path=Name}" />
```

```
<TextBlock Text="{x:Bind Path=Surname}" />
<TextBlock Text="{x:Bind Path=BirthDate}" />
</StackPanel>
</DataTemplate>
```

Now that we have set the **DataType** property, we can change **Binding** with **x:Bind** in all the markup expressions inside **DataTemplate**.

Event binding

A new feature supported by **x:Bind** is event binding, which means that you can connect events exposed by the controls with the function you want to execute using a binding expression, instead of a traditional event handler. For example, you can implement the **Click** handler of **Button** using the following expression:

```
<Button Click="{x:Bind DoSomething}" Content=
    "Do something" />
```

Then, in the code-behind of your page or user control, you just need to declare a method called **DoSomething**, as shown in the following example:

```
public void DoSomething()
{
    Debug.WriteLine("Do something");
}
```

The method can be parameter-less (as shown in the previous example), or it can match the same signature of the original event handler. This is another example of a valid implementation:

```
public void DoSomething(object sender, RoutedEventArgs
e)
{
    Debug.WriteLine("Do something");
}
```

Collection binding

Thanks to the **x:Bind** markup expression, you can also easily connect your UI controls with specific items included in a collection by specifying its index. For example, suppose you have the following collection declared in the code-behind:

```
public ObservableCollection<Person> People { get; set; }
public MainWindow()
{
    this.InitializeComponent();
    People = new ObservableCollection<Person>
    {
        new Person { Name = "John", Surname = "Doe",
            BirthDate = new DateTime(1980, 10, 1) },
        new Person { Name = "Michael", Surname =
            "Green",
            BirthDate = new DateTime(1975, 4, 3) }
    };
}
```

If you want to display some information about the second person in the list, you can use the **x:Bind** expression, as shown in the following example:

```
<TextBlock Text="{x:Bind People[1].Name}" />
```

Another type of collection supported by the **x:Bind** markup expression is dictionaries. In this case, you can use a similar syntax to the previous one; you just need to specify the key instead of the index. For example, let's say you have the following dictionary declared in the code-behind:

```
public Dictionary<string, Person> PeopleDictionary {
    get;
    set; }
public MainWindow()
{
```

```
this.InitializeComponent();
PeopleDictionary = new Dictionary<string, Person>
{
    { "John", new Person { Name = "John", Surname =
        "Doe", BirthDate = new DateTime(1980, 10, 1) }
    },
    { "Michael", new Person { Name = "Michael",
        Surname = "Green", BirthDate = new
        DateTime(1975,
            4, 3) } }
};
}
```

If you want to display some information about the second item in **Dictionary** (which is identified by the **Michael** key), you can use the following binding expression:

```
<TextBlock Text="{x:Bind PeopleDictionary['Michael']
    .Name}" />
```

Functions

One of the most powerful features exposed by **x:Bind** is function support, which enables you to create complex expressions that can manipulate data coming from the source of the binding channel before it's received by the target.

With standard binding, this feature can be achieved using converters; however, they have some downsides:

- They are evaluated at runtime, adding performance overhead.
- They don't support more than one parameter.
- They are harder to maintain, since they must be declared as XAML resources.

Thanks to functions, you can pass a method declared in the code-behind (or in a static class) as body of a binding expression. For example,

let's reuse **DataTemplate** that we previously created in this section:

```
<ListView>
  <ListView.ItemTemplate>
    <DataTemplate x:DataType="local:Person">
      <StackPanel>
        <TextBlock Text="{x:Bind Path=Name}" />
        <TextBlock Text="{x:Bind Path=Surname}" />
        <TextBlock Text="{x:Bind Path=BirthDate}"
      />
    </StackPanel>
  </DataTemplate>
</ListView.ItemTemplate>
</ListView>
```

When you use this template to render a collection of **Person** objects, you will get a similar outcome:

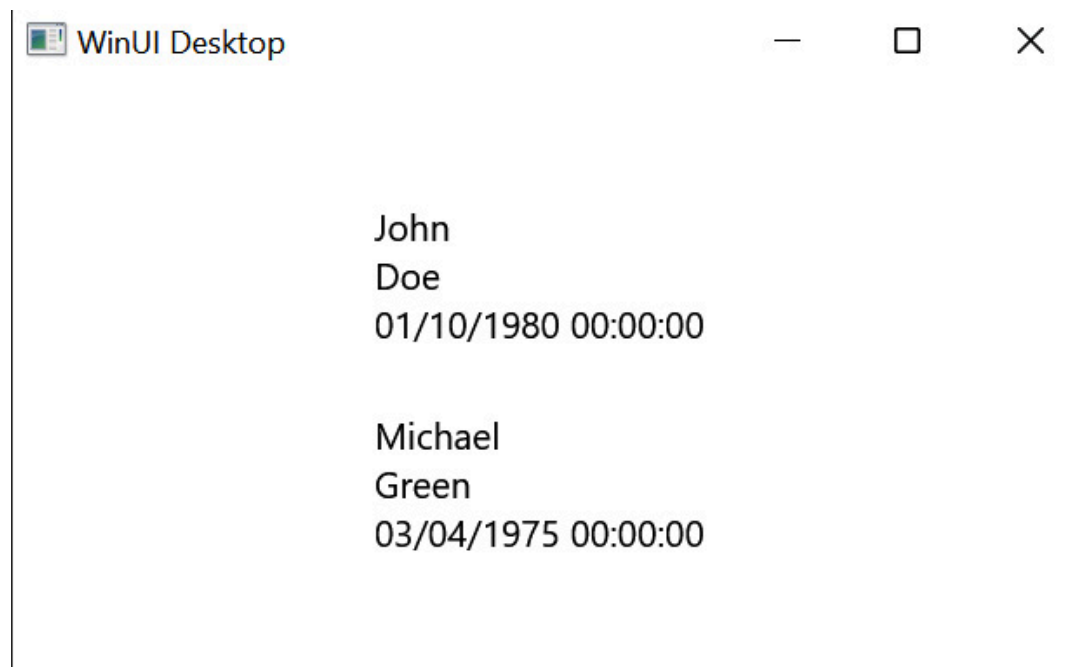


Figure 3.1 – A ListView control that displays a list of people

As you can see, we are displaying unnecessary information – since the **DateTime** property is mapped with the birth date of the person, we don't need to display the time as well. Thanks to functions, we just need to add a new method to our **Person** class to achieve the goal:

```
public class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public DateTime BirthDate { get; set; }
    public string ConvertToShortDate(DateTime dateTime)
    {
        return dateTime.ToShortDateString();
    }
}
```

Now, we can change our binding expression in **DataTemplate** to use this new function:

```
<DataTemplate x:DataType="local:Person">
    <StackPanel Margin="12">
        <TextBlock Text="{x:Bind Path=Name,
            Mode=OneWay}" />
        <TextBlock Text="{x:Bind Path=Surname,
            Mode=OneWay}" />
        <TextBlock Text="{x:Bind
Path=ConvertToShortDate
            (BirthDate), Mode=OneWay}" />
    </StackPanel>
</DataTemplate>
```

Now, our list won't contain the time anymore when it displays the birth date of the person:

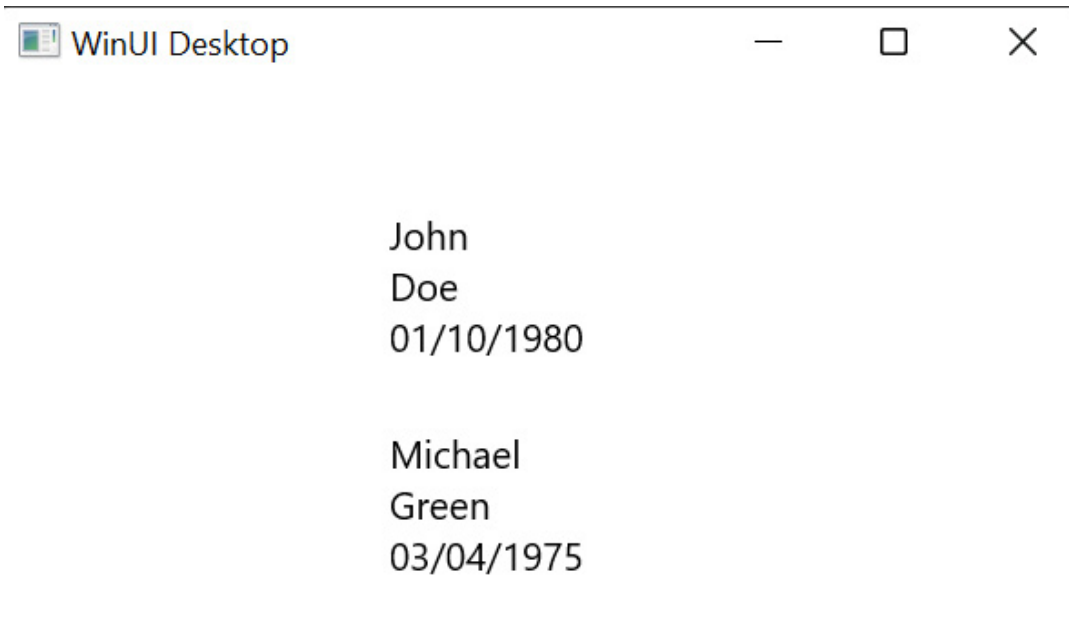


Figure 3.2 – DataTemplate now displays only the date, thanks to the `x:Bind` function support

In a more complex application, however, you might not appreciate this code, since it forces you to change the definition of the entity (in this case, the **Person** class) to add extra code that is specific to the way the data is displayed (the function to format the date). Thanks to property path support, you can easily move this code to a static class, as shown in the following example:

```
public static class DateTimeHandler
{
    public static string ConvertToShortDate(DateTime
        dateTime)
    {
        return dateTime.ToShortDateString();
    }
}
```

Then, you can use the familiar dot (.) notation to use this class in the binding expression, as shown in the following snippet:

```
<DataTemplate x:DataType="local:Person">
    <StackPanel Margin="12">
```

```

        <TextBlock Text="{x:Bind Path=Name,
            Mode=OneWay}" />
        <TextBlock Text="{x:Bind Path=Surname,
            Mode=OneWay}" />
        <TextBlock Text="{x:Bind
Path=local:DateTimeHandler
        .ConvertToShortDate(BirthDate), Mode=OneWay}"
/>
    </StackPanel>
</DataTemplate>

```

In this case, however, since **DateTimeHandler** is a different class than **Person**, we need to specify the full namespace (**local:DateTimeHandler**).

Functions enable us to also use standard C# system functions directly in markup without having to build a custom one. For instance, we can achieve the same goal of the earlier example without building a custom **DateTimeHandler** class, directly using the **String.Format()** function included in C# instead, as shown in the following snippet:

```

<Page
    x:Class="App1.MainPage"
    xmlns:sys="using:System"
    mc:Ignorable="d">
    <Page.Resources>
        <DataTemplate x:Key="PersonTemplate"
            x:DataType="local:Person">
            <StackPanel Margin="12">
                <TextBlock Text="{x:Bind Path=Name,
                    Mode=OneWay}" />
                <TextBlock Text="{x:Bind Path=Surname,
                    Mode=OneWay}" />
                <TextBlock Text="{x:Bind
                    Path=sys:String.Format('{0:dd/MM/yyyy
    }',

```



```
        BirthDate}}" />
    </StackPanel>
</DataTemplate>
</Page.Resources>
</Page>
```

First, you have to map the **System** namespace to a XAML namespace (in the previous example, we called it **sys**). Then, you can use the **String.Format()** function to format the date in the same way you would do in C#. The outcome of the previous snippet will be the same as the one with the custom function – only the date will be displayed, without the time.

Another powerful feature of functions is the ability to support multiple parameters. With converters, you are limited to supplying a single parameter through the **ConverterParameter** function, as shown in the following example:

```
<TextBlock Foreground="{Binding Path=BirthDate,
    Converter={StaticConverter BudgetColorConverter}
    ,ConverterParameter=20}"}"
```

Additionally, since **ConverterParameter** isn't a dependency property, you can't use a binding expression, meaning that you can't easily pass as a parameter another property of the same binding context (for example, another property of the **Person** class).

Thanks to functions, both scenarios can be easily achieved, since, in the end, a function is nothing more than a C# method, so you can have as many parameters as you need and use **x:Bind** to assign them.

To show this feature in action, let's add two new properties to our **Person** class, **Budget** and **Stocks**:

```
public class Person
{
    public string Name { get; set; }
```

```
public string Surname { get; set; }  
public DateTime BirthDate { get; set; }  
public double Budget { get; set; }  
public int Stocks { get; set; }  
}
```

Now, let's assume that, in our **DataTemplate**, we want to show the item's background with a different color so that our users can quickly understand the financial status of the person. However, in our scenario, the financial status is based on the budget and the number of stocks, so we need to do some math first. Let's create a new static class that supports our requirement:

```
public static class BudgetColorHandler  
{  
    public static SolidColorBrush GetBudgetColor(double  
        budget, double stocks)  
    {  
        if (budget < 100 || stocks <10 )  
        {  
            return new SolidColorBrush(Colors.Red);  
        }  
        else  
        {  
            return new  
SolidColorBrush(Colors.Green);  
        }  
    }  
}
```

The class exposes a function called **GetBudgetColor()**, which returns a **SolidColorBrush** object based on two parameters – **budget** and **stocks**. If they are below a certain threshold, the returned color will be red; otherwise, it will be green.

Now, we can customize our **DataTemplate** to use this new function:

```
<DataTemplate x:Key="PersonTemplate"
  x:DataType="local:Person">
  <StackPanel Margin="12" Background="{x:Bind
    local:BudgetColorHandler.GetBudgetColor(Budget,
    Stocks)}">
    <TextBlock Text="{x:Bind Path=Name, Mode=
      OneWay}" />
    <TextBlock Text="{x:Bind Path=Surname,
      Mode=OneWay}" />
    <TextBlock Text="{x:Bind Path=BirthDate)}" />
  </StackPanel>
</DataTemplate>
```

As you can see, compared to the first example based on converters, thanks to **x:Bind**, we have easily been able to do the following:

- Create a binding expression calculated based on two parameters.
- Create a binding expression where the two parameters aren't static values but are actual properties of the **Person** class.

The following screenshot shows the outcome of the previous example:

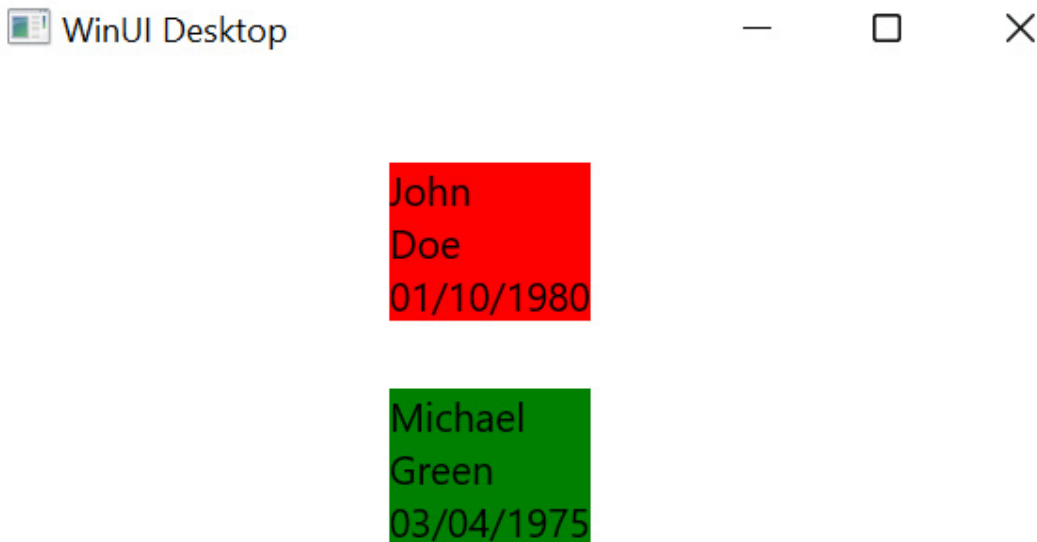


Figure 3.3 – DataTemplate using a function with multiple parameters to apply a different background color

Lastly, one of the most powerful features of functions is that they are all re-evaluated every time one of the parameters changes, as long as they properly implement the **INotifyPropertyChanged** interface so that they can notify the binding channel that its value has changed. In the previous example, changing the value of the **Budget** or **Stocks** property will cause a new evaluation of the **GetBudgetColor()** function, potentially leading to a change in the background color applied to the item.

Now that we have learned how to improve the usage of binding in WinUI thanks to the **x:Bind** keyword, we can move on to learn more about the next difference between WPF and WinUI – the usage of the dispatcher.

Using the dispatcher

WinUI, like WPF, uses a single-thread model to manage the UI. This means that, in your code, you can interact with the controls included in your window only from the UI thread. Consider, for example, the following code:

```
Task.Run(() =>
{
    //do some heavy work
    myTextBlock.Text = result;
});
```

By using **Task.Run()**, we are forcing the code to be executed on a background thread. However, if you run this snippet, you will get an exception, as shown in the following screenshot:

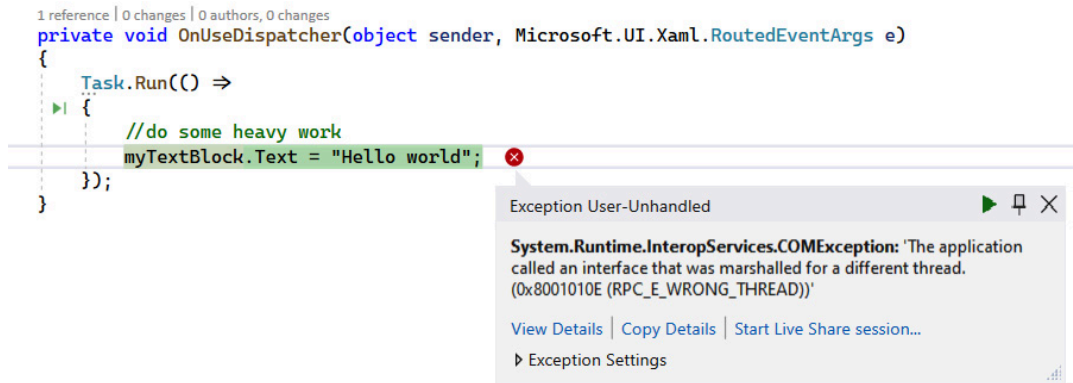


Figure 3.4 – The exceptions raised when you try to access the UI from a different thread

This is expected, since we're trying to access UI control from a background thread. To manage these scenarios, all the .NET UI platforms use the concept of a dispatcher, which, as the name suggests, is able to dispatch a specific action from a different thread to the UI thread.

In WPF, the dispatcher is implemented by the **Dispatcher** class, which exposes methods such as **Invoke()** or **BeginInvoke()**.

In WinUI, it's implemented instead by the **DispatcherQueue** class, which belongs to the **Microsoft.UI.Dispatching** namespace. To execute a specific action on the UI thread, you use the **TryEnqueue()** method, as shown in the following example:

```
await Task.Run(() =>
{
    //do some heavy work
    Microsoft.UI.Dispatching.DispatcherQueue.
        TryEnqueue(() =>
        {
            myTextBlock.Text = "Hello world";
        });
});
```

This code won't raise an exception anymore, since the line of code that interacts with the UI (updating the **Text** property of a **TextBlock** control) is added to the dispatcher's queue.

The previous code example works fine if you need to use the **DispatcherQueue** class in a code-behind class, since, in this scenario, WinUI already knows which is the UI thread. If you need to use the dispatcher from a regular class, you will have to obtain first the proper reference for the current thread by using the **GetForCurrentThread()** method, as shown in the following example:

```
var dispatcher = Microsoft.UI.Dispatching.  
    DispatcherQueue.GetForCurrent  
    Thread();  
Task.Run(() =>  
{  
    dispatcher.TryEnqueue(() =>  
    {  
        Name = "This message has been updated from a  
background  
        thread";  
    });  
});
```

This usage of the dispatcher, however, could lead to a few challenges when you need to use it in combination with asynchronous APIs. Let's see how we can improve the code.

Improving dispatcher usage in asynchronous scenarios

The **DispatcherQueue** object has a downside – it doesn't provide support for asynchronous operations. This means that, if you need to execute some code only when the operation you have dispatched has been completed, you will be forced to split your flow into two parts and continue the code execution inside the dispatcher, as shown in the following example:

```
Task.Run(() =>  
{
```

```
//do some heavy work
Microsoft.UI.Dispatching.DispatcherQueue
    .TryEnqueue(() =>
    {
        myTextBlock.Text = result;
        //continue the work
    });
});
```

Thanks to the Windows Community Toolkit

(<https://docs.microsoft.com/en-us/windows/communitytoolkit/>), an open source project by Microsoft, we can improve the usage of the dispatcher in asynchronous scenarios, thanks to an extension method.

The first step is to install the **CommunityToolkit.WinUI** NuGet package in your WinUI application. Then, you must add the following namespace in your class:

```
using CommunityToolkit.WinUI;
```

Now, you can use the **EnqueueAsync()** method instead of **TryEnqueue()** to dispatch actions to the UI thread. This method implements the **Task** class, so you can use the **async - await** pattern to invoke it, as shown in the following example:

```
Task.Run(async () =>
{
    //do some heavy work
    await DispatcherQueue.EnqueueAsync(() =>
    {
        myTextBlock.Text = result;
    });
    //continue the work
});
```

As you can see, we don't have to continue the work inside the dispatcher's action anymore. By using the **await** keyword, we can be

sure that the rest of the code will be executed only when the action we have enqueued has been completed.

You can also return a value from the dispatched action by using the **EnqueueAsync<T>()** variant of the method, as shown in the following example:

```
string result = await DispatcherQueue
    .EnqueueAsync<string>(() =>
    {
        return "Hello world";
    });
var output = $"Result: {result}";
```

Now that you have learned how to properly use the dispatcher, we can explore the last important difference between WPF and WinUI – localization.

Localizing your applications

Supporting multiple languages is a key requirement for applications, especially in the enterprise space. Offering an application in the native language of our users improves the user experience and also makes it easier for people who might not be familiar with the technology to use.

WinUI, as with other UI platforms, offers a way to create a UI that can be easily localized by defining placeholders that, at runtime, are replaced with strings taken from a localization file that matches the language of the user.

Localization is one of the areas that has been significantly upgraded from WPF, thanks to a new resource management system called MRT Core, which, other than supporting localization, also enables us to eas-

ily load custom resources, such as a different image based on the scaling factor of a device or based on the Windows theme that we set.

The starting point to enable localization is to add a resource file in your application, which is based on the **.resw** extension (unlike in WPF, which used files with the **.resx** extension).

These files simply contain a collection of keys with their corresponding value:

- The key is used as a placeholder, and it will be referenced in our UI and code.
- The value is the localized text that we want to display as a replacement for the placeholder.

As a developer, you won't have to manually load one resource file or another based on the language of the user. Windows will take care of loading the proper resource file. It's enough for you to add resources to your project following one of the available naming conventions. One approach is to create a folder called **Strings** and, inside it, create one subfolder for each language you need to support, using the language code (for example, **en-US** or **it-IT**). Then, inside each folder, include a file called **Resources.resw**, which will include all the localized resources for the corresponding language code. The following screenshot shows what a project that uses this naming convention looks like:

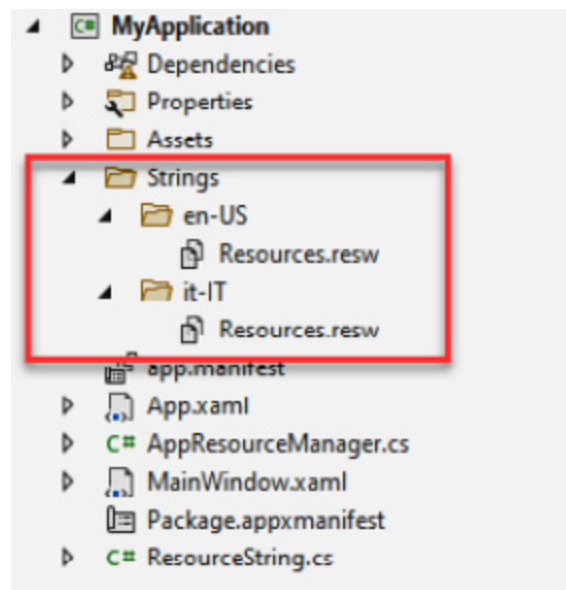


Figure 3.5 – An application that supports localization in English and Italian

This is the suggested approach, since it helps to keep the project's structure clean and well-organized. However, if you prefer, you can also use another approach, which is keeping all the resource files at the root folder of the project but adding the word **lang** as a suffix (after the filename), followed by the language code the file is related to. The following screenshot shows an example of this approach:

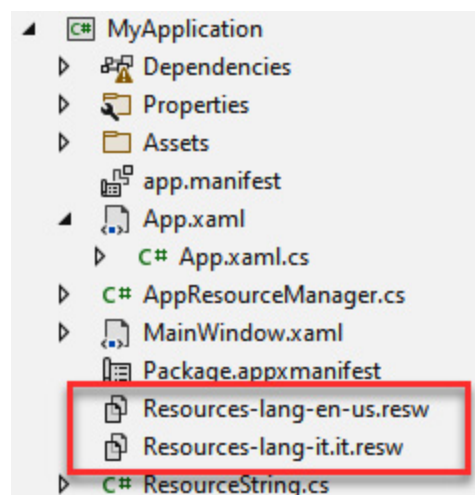


Figure 3.6 – An application that supports localization in English and Italian but using a different naming convention

Both files are placed in the root folder, but they're called **Resources-lang-en-us.resw** (which contains the strings localized in English) and

Resources-lang-it-it.resw (which contains the strings localized in Italian).

Regardless of the approach you prefer, the way to create a new resource file is the same. You right-click on your project (or on a specific folder, in case you opt in for the approach), choose **Add | New item**, and select the template called **Resources file (.resw)**. Once you have added a resource file, Visual Studio will automatically open the resource editor, as in the following screenshot:

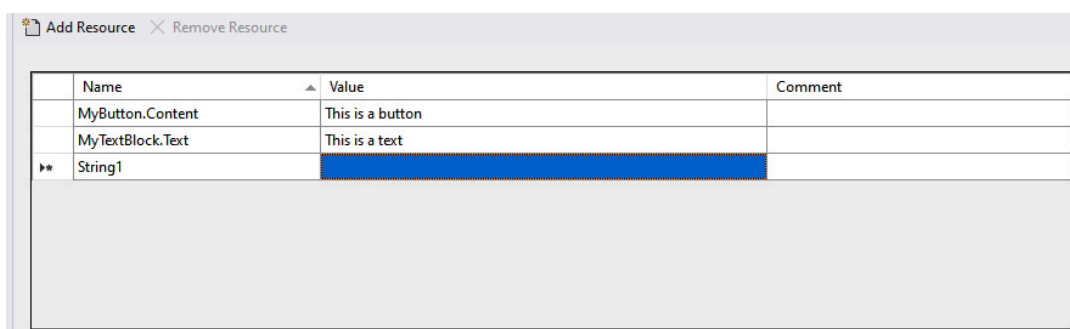


Figure 3.7 – The resource editor included in Visual Studio

As mentioned at the beginning of the section, a resource file is nothing more than a collection of names with their corresponding values. However, before starting to fill the file with one or more resources, it's important to understand how to use them.

Translating the user interface

WinUI makes it easy to use resources to localize the UI, thanks to a special property called **x:Uid**, which can be set for any XAML control. Let's say, for example, that we want to localize the label of a **Button** control, which is defined by the **Content** property. First, we need to assign the name of the resource we want to use to the **x:Uid** property, as shown in the following example:

```
<Button x:Uid="MyButton" />
```

Now, we can add the resource in our resources file, using the **<name of the resource>.<name of the property>** syntax. In our case, since we

want to localize the label (which, for a **Button** control, is stored inside the **Content** property), we're going to create a resource named **MyButton.Content**. As the value, we must add the text that we want to display in the language the file refers to. Now, we have to repeat this process for every other resource file in our project. At the end, all the resources files must have a resource called **MyButton.Content** with, as the value, the translated label.

As you can see, this approach is more powerful than the one included in WPF, since it enables us to not only localize the label but also other properties based on the user's language. For example, let's say that we want the **Width** property of the **Button** control to be bigger when the application is localized in Italian, since the Italian label takes more space. We just need to add, only in the Italian resources file, a resource called **MyButton.Width** with, as the value, the size we want to set.

Handling translation in code

There are scenarios in which you need to retrieve a translated string directly in the code of the application (for example, when you need to localize the message of a popup dialog). In this scenario, things are slightly more complicated. Let's understand the reasoning a bit more.

When you enable localization using the special **x:Uid** property, WinUI generates an instance of a class called **ResourceManager** under the hood, which is responsible for properly loading the resources file and retrieving the correct resource based on the value of the property. Unfortunately, at the time of writing, WinUI doesn't expose the **ResourceManager** instance, which is generated behind the scenes, making it impossible to use it to load translations in code.

As such, we have to create and maintain our own instance of the **ResourceManager** class and use it to retrieve localized strings from a resource file. The easiest way to achieve this goal is to create a helper

class, which we're going to call **AppResourceManager**, that will take care of managing the **ResourceManager** instance. Here is the full implementation:

```
public sealed class AppResourceManager
{
    private static AppResourceManager instance = null;
    private static ResourceManager _resourceManager =
null;
    private static ResourceContext _resourceContext =
null;
    public static AppResourceManager Instance
    {
        get
        {
            if (instance == null)
                instance = new AppResourceManager();
            return instance;
        }
    }
    private AppResourceManager()
    {
        _resourceManager = new ResourceManager();
        _resourceContext = _resourceManager
.CreateResourceContext();
    }
    public string GetString(string name)
    {
        var result = _resourceManager.MainResourceMap
.GetValue($"Resources/{name.Replace(".",
"/")})",
        _resourceContext).ValueAsString;
        return result;
    }
}
```

Here is the most essential information to know about this class:

- To avoid having too many instances of the **ResourceManager** class, which might lead to memory leaks, we use the singleton approach to make sure that every class in our project is always going to reuse the same instance.
- We expose a method called **GetString()**, which returns the translated string based on the name of the resource that we provide. These resources are stored inside the **MainResourceMap** collection of the **ResourceManager** instance. The naming convention to retrieve a resource is *Resources/<name of the resource>/<name of the property>*. However, since in our resource file a resource is defined as *<name of the resource>.<name of the property>*, we have to apply a string replacement.
- You can see that we are also passing a **ResourceContext** object to retrieve a localized string. This object includes the whole application's context (language, scaling factor, theme, and so on), which is set automatically by the operating system. As we're going to see later, we can also use the context to override the default settings.

Once we have added this class to our project, we can simply retrieve a resource in a class using the following code:

```
var translatedLabel = AppResourceManager  
    .Instance.GetString("MyButton.Content");
```

A better way to manage localization

The **x:Uid** property helps to implement localization in an easier way, compared to WPF. However, it still has a few limitations. For example, it makes it impossible to reuse the same resource for multiple controls, since each resource name includes a reference to **x:Uid** (and you can't have two controls on the same page with the same **x:Uid**). Another critical scenario is enabling users to choose their own preferred language. By default, WinUI will load the resource file based on the dis-

play language configured in the Windows settings. You can find this property in the Settings application by following the **Time & Language | Language & Region | Windows display language** path, as in the following screenshot:

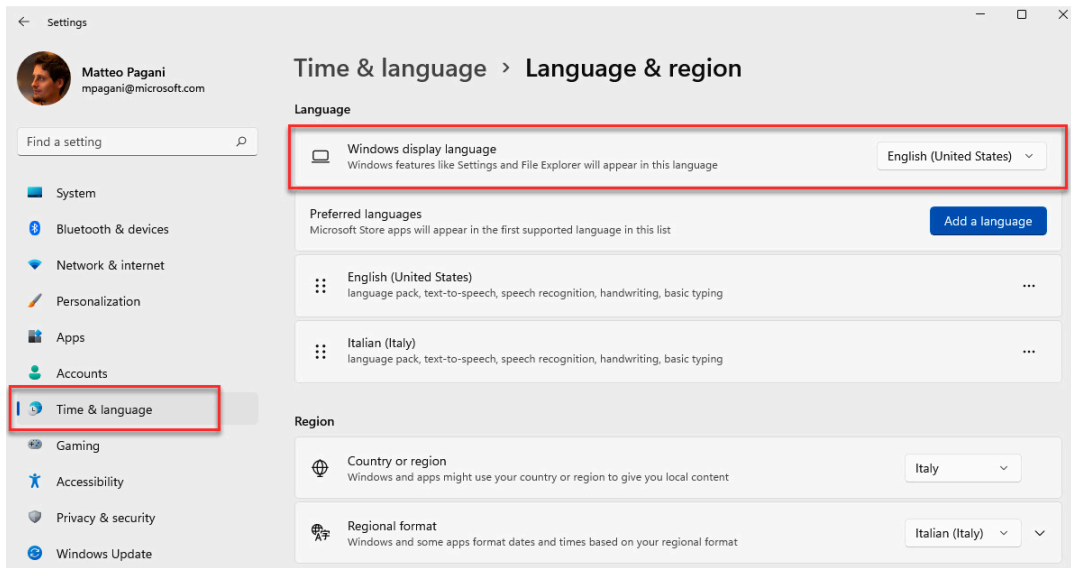


Figure 3.8 – The settings in Windows 11 to customize the Windows display language

However, there are scenarios in which you might want to allow your users to set a different language than the one used for the operating system. In this case, we can leverage the **ResourceContext** class to force a different context, instead of using the default one. For example, if we want to force our application to use resources in Italian, we can customize the public constructor of the **AppResourceManager** class that we previously created in the following way:

```
private AppResourceManager()  
{  
    _resourceManager = new ResourceManager();  
    _resourceContext =  
        _resourceManager.CreateResourceContext();  
    _resourceContext.QualifierValues["Language"] = "it-  
IT";  
}
```

By overriding the **Language** qualifier, our **ResourceManager** instance will start to retrieve resources for the specific chosen language, instead of the default one provided by the operating system.

However, there's a catch if you're using the **x:Uid** property to handle localization. Remember that, in a WinUI app, we don't have a way to retrieve the **ResourceManager** instance that is automatically used by the **x:Uid** property, so we had to create a new one inside the **AppResourceManager** class. This means that we can't set this property on the built-in instance, which will lead to a disconnection between UI and code:

- In code, since we're using the **AppResourceManager** class, we'll start to retrieve strings in the language we have forced.
- In XAML, since we're using the **x:Uid** property, WinUI will continue to use the default display language in Windows to choose the resource to load.

As such, we can enable a better way to manage localization (inspired by the following blog post from a Microsoft engineer:

<https://devblogs.microsoft.com/ifdef-windows/use-a-custom-resource-markup-extension-to-succeed-at-ui-string-globalization/>) by using a custom markup expression, which we're going to use as a replacement for the **x:Uid** property. First, let's create the markup expression by adding the following class to our project:

```
[MarkupExtensionReturnType(ReturnType =
typeof(string))]
public sealed class ResourceString : MarkupExtension
{
    public string Name
    {
        get; set;
    }
    protected override object ProvideValue()
```



```
{  
    string value = AppResourceManager  
        .Instance.GetString(Name);  
    return value;  
}  
}
```

As you can see, the **ProvideValue()** method (which defines the value to return when the markup expression is used in XAML) uses the same **AppResourceManager** instance that we are using across our code, giving us the consistency we need across the entire application.

Now, we can remove the **x:Uid** property from our **Button** control in the XAML page and replace it with our markup expression, as shown in the following example:

```
<Button Content="{local:ResourceString  
    Name=MyButton.Content}" />
```

An important difference, compared to **x:Uid**, is that we need to use the markup expression to explicitly set the property we want to change; we can't rely anymore on the *<name of the resource>. <name of the property>* naming convention. In the previous example, since we're localizing the **Button** label, we're using the **ResourceString** markup expression to set the **Content** property.

This approach has multiple advantages:

- We can simplify resource files, since we can use a plain string to define a resource (for example, **MyButtonLabel**).
- We can reuse the same resource with multiple controls.
- Since we don't have a reference to the XAML control in the resource name anymore, it's easier to do refactoring if we need to change the UI.

Support localization in a packaged app

If you're using the packaging approach to distribute your application, there are two extra steps you need to take:

- Declaring the supported languages in the application manifest
- Defining what the default language is for your application

To accomplish the first task, right-click on the **Package.appxmanifest** file included in your project, choose **View Code**, and locate the following entry:

```
<Resources>
  <Resource Language="x-generate"/>
</Resource>
```

Delete it and replace it with a new **Resource** entry for each language you want to support and set, as the value of the **Language** attribute, the language code. For example, since in our previous example we supported English and Italian, our entry will look like this:

```
<Resources> a method declared in the code-behind (or in
a
static class)
  <Resource Language="en-US"/>
  <Resource Language="it-IT"/>
</Resource>
```

The second task can be accomplished from the manifest as well. However, this time, we can use the visual editor, so it's enough to double-click on the **Package.appxmanifest** file in Visual Studio and to move to the **Application** tab. You will find a property called **Default language**, which, by default, is set to **en-US**. This is the suggested configuration – if you are using Windows in a language that you don't directly support, the application will be displayed in English. However, if your application targets a specific market (for example, it's meant to be used only in Germany), you are free to change this value to a more appropriate one.

Summary

In this chapter, we have explored the key differences between WPF and WinUI when it comes to building the UI of our applications. Compared to a Windows Forms developer, we have a significant advantage, since many of the basic concepts behind WinUI are the same ones as WPF because they are both based on XAML as a markup language. However, WinUI is a newer and more modern UI framework, and as such, we can enjoy some new powerful features, such as the new `x:Bind` expression to enable compiled bindings and the new localization approach.

Thanks to the skills acquired in this chapter, you now have a path forward to modernize your existing WPF applications by creating a new and modern UI experience, thanks to the new WinUI features.

However, when you start building an application based on WinUI, you will find other differences compared to WPF. For example, the way you enable navigation is different, there are many new controls that you can leverage to build the UI, and WinUI has a much more powerful rendering system that supports creating beautiful animations.

We didn't cover these features in this chapter though, as they aren't specific for WPF developers, but they do affect Windows Forms developers. As such, we will discuss them in [*Chapter 5, Designing Your Application*](#), which is entirely dedicated to designing great UIs with WinUI.

The next chapter will be dedicated to developers who already have experience in building Windows applications with Universal Windows Platform and who would like to move them forward by adopting WinUI and the Windows App SDK.

Questions

1. If you need to format a string that is managed by a binding channel created with **x:Bind**, the only option is to create a custom function – true or false?
2. When you use the **DispatcherQueue** class, there's no way to wait for the action dispatched to the UI thread to be completed before performing other tasks – true or false?
3. In a WinUI application, you must detect in code what the current display language in Windows is and load the proper resource file accordingly – true or false?